# 位运算

2026年4月18日 14:12

1. 基础位运算

& → 有0就为0

| → 有1就为1

^ → 无进位相加 or 相同为1，相异为0

```
  0 1 1 0 1
  1 0 1 0 0
& 0 0 1 0 0
  1 1 1 1 0 1
  1 1 1 0 0 1
```

2. 给定一个数n，确定它的二进制表示中第x位是0还是1

明确 高位 ← 低位

$\boxed{1 \& (n >> x)}$ 最低位

```
0 0 1 0 1
      x
不管
  & 1
  0 1
```

3. 将1个数n的二进制表示的第x位改为1

其他位不变

```
0 1 0 1 0 1
x
0 1 0 1 1 0 1
```

$\boxed{n \mid = (1 << x)}$ 移到第x位

移到这

4. 将1个数n的二进制表示的第x位改为0

```
0 1 0 1 1 0 1
0 1 0 1 0 0 1
0 0 0 0 1 0 0
~ 1 1 1 1 0 1 1
```

先移到这

现在要 & = 0 相当于为 0

$\boxed{n \& = (\sim(1 << x))}$

先把1移过去，然后取反就有0了

并且保证其他位要不变，只能全为1

5. 提取一个数n二进制表示中最右侧的1

提取

```
  0 1 1 0 1 0 1 1 0 0 0
  0 0 0 0 0 0 0 1 0 0 0
~ 1 0 0 1 0 1 0 0 1 1 1
+1
  1 0 0 1 0 1 0 1 0 0 0
  0 0 0 0 0 0 0 1 0 0 0
```

分区操作

-n 以最右侧1(包含)为界

右：不变

左：取反

$\boxed{n \& (-n)}$

6. 干掉一个数n二进制中最右侧的1

```
  0 1 1 0 1 0 1 1 0 0 0
-1
  0 1 1 0 1 0 1 0 1 1 1
  0 1 1 0 1 0 1 0 0 0
```

(n-1) 以最右侧1(包含)为界

右：取反

左：不变

$\boxed{n \& (n-1)}$

7. 异或^的运算规律

$a \wedge 0 = a$

$a \wedge a = 0$ 消消乐

$a \wedge b \wedge c = a \wedge (b \wedge c)$

## 题型一：状态压缩/位图思想解决

一个int变量32位，把它每一个比特位都当成一个储物格子

0代表空，1代表有东西。

给一个字符串，判断里面有没有重复的字符

26个字母，int 当备忘录，遇到'a'，就把第0个格子占了，若下次再遇到'a'，碰到被占了的'a'说明重复！

[面试题 01.01. 判定字符是否唯一 - 力扣（LeetCode）](https://leetcode.cn/problems/is-unique-lcci/)

```cpp
class Solution {
public:
    bool isUnique(string astr) {
        int bitMap = 0;
        for(auto ch : astr) {
            // 找到在位图中的对应位置
            int pos = ch - 'a';
            // 判断这个位置比特位是否存在元素
            if(((bitMap >> pos) & 1) == 1) return false;
            // 不存在，添加
            else bitMap |= (1 << pos);
        }
        return true;
    }
};
```

## 题型二：模拟底层硬件电路

不用 + 模拟加法运算

加法：^ 无进位相加，1^1=0，1^0=0

进位：& 后 <<1，只有1&1才产生进位

```
  1 0 1
  0 0 1
& 1 0 1
```

异或算本位

& 后<<1 算进位

不断循环，直到无进位

[371. 两整数之和 - 力扣（LeetCode）](https://leetcode.cn/problems/sum-of-two-integers/)

```cpp
class Solution {
public:
    int getSum(int a, int b) {
        while(b) {
            // 无进位相加
            int noCarry = a ^ b;
            // 找进位，防止左移到符号位溢出
            unsigned int carry = (unsigned int)(a & b) << 1;
            // 不断循环
            a = noCarry;
```

```
            a = noCarry;
            b = carry;
        }
        return a;
    }
};
```

## 题型三：异或消消乐

$a \wedge a = 0$

$a \wedge 0 = a$

$a \wedge b \wedge c = a \wedge (b \wedge c)$

**丢失的数字**

造一个完整的 0 到 n 的数组。把【完整的数组】和【残缺的数组】混在一起全部异或。这时候，没丢的数字都出现了 2 次（抵消成 0），那个丢失的数字只出现了 1 次，直接现原形！

### 137. 只出现一次的数字 II

已解答

中等 相关标签 相关企业 Aa

给你一个整数数组 `nums`，除某个元素仅出现 **一次** 外，其余每个元素都恰出现 **三次**。请你找出并返回那个只出现了一次的元素。

你必须设计并实现线性时间复杂度的算法且使用常数级空间来解决此问题。

对于任意一个比特位，出现了3次的数字，在这1位贡献1的数量一定是3的整数倍！

把所有数字的某一位的1全部加起来，模3，出现两种情况：

==0：说明只出现一次的数字在这一位贡献为0

==1：说明是3n+1，只出现一次的数字在这一位贡献为1

% 3 就可以把出现三次的数字过滤掉，我们只需要根据 %3 的结果组装只出现一次的数字

137. 只出现一次的数字 II - 力扣（LeetCode）

```
class Solution {
public:
    int singleNumber(vector<int>& nums) {
        int ans = 0;
        for(int i = 0; i < 32; i++) {
            int sum = 0;
            for(int x : nums) {
                // 统计所有数的特定位对一的贡献
                if(((x >> i) & 1) == 1) sum++;
            }
            // 出现三次的数字被过滤掉了，只剩下出现一次的数
            sum %= 3;
            // 找到1对应的比特位
            if(sum == 1) ans |= (1 << i);
        }
         return ans;
    }
};
```

## 题型四：找边界

提取两个不同数字的差异位，一般都是最右边第一个1：

lowBit操作： nums & (-num)

把数组一分为2

### 260. 只出现一次的数字 III

已解答

中等 相关标签 相关企业 Aa

给你一个整数数组 `nums`，其中恰好有两个元素只出现一次，其余所有元素均出现两次。 找出只出现一次的那两个元素。你可以按 **任意顺序** 返回答案。

你必须设计并实现线性时间复杂度的算法且仅使用常量额外空间来解决此问题。

思路：

成对的异或在一起全变为0

只剩两个单独出现的a和b一定不相等

a^b，现在需要把这两个数字拆开

eg：3∧5

0 1 1

1 0 1

1 1 0

哪个1位出现了0，说明3和5这1位一定相同

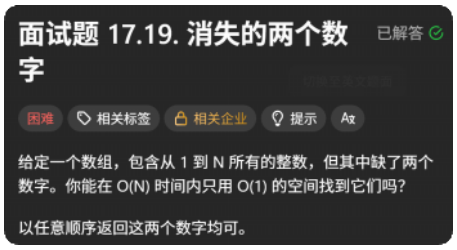
只需找个分界线 → 现成的 lowbit

a∧b 最右边的 1 → a和b在这一位上不同

260. 只出现一次的数字 III - 力扣（LeetCode）

```
class Solution {
public:
    vector<int> singleNumber(vector<int>& nums) {
        unsigned int num = 0;
        for(int x : nums) num ^= x;
        // 找到a ^ b的分界线
        int diff = num & (-num);
        // 分成两组消消乐
        int a = 0, b = 0;
        for(int x : nums) {
            if((x & diff) != 0) a ^= x;
            else b ^= x;
        }
        return {a,b};
    }
};
```

面试题 17.19. 消失的两个数字 - 力扣（LeetCode）




1. 把名单上的（12345）和现场的人（135）全部关进屋子里进行异或
   消消乐最后只会剩下2^4

2. 怎么分开2^4?
   a. 找边界，lowbit
   b. 分两组重新异或，这个目的是把2和4分开

面试题 17.19. 消失的两个数字 - 力扣（LeetCode）

```
class Solution {
public:
    vector<int> missingTwo(vector<int>& nums) {
        // 1. 消消乐找到缺失的两个数字的异或
        int num = 0;
        for(int x : nums) num ^= x;
        for(int i = 0; i <= nums.size() + 2; i++) num ^= i;
        // 2. 找到两个数字的分解线，直接lowbit省事
        int diff = num & (-num);
        // 3. 分两组异或，把两个数字分开
        int a = 0, b = 0;
        for(int x : nums) {
            if((x & diff) != 0) a ^= x;
            else b ^= x;
        }
        for(int i = 0; i <= nums.size() + 2; i++) {
            if((i & diff) != 0) a ^= i;
            else b ^= i;
        }
        return {a,b};
    }
};
```